

Phenotype Portfolio CVP & Semantic Compression Audit

Fresh GitHub pass vs. the 11 September 2026 snapshot

Current state: 14 September 2026

*Focus: installed Current Viable Products, reversible portfolio reduction,
semantic codebase compression, and recursive applet composition*

How to read this audit

This is a fresh remote-GitHub audit and an engineering-plan update, not a declaration that every local worktree, chat, release artifact, or product journey has been independently executed. The evidence classes below are kept separate so commit messages and automation claims do not silently become product facts.

Evidence class	Meaning
OBSERVED	Directly visible in current GitHub repository metadata, commit graph, PR metadata, Actions, or release metadata.
REPORTED	Claim in a commit/PR/workflow description that was not rerun independently in this pass.
INFERRED	Engineering conclusion derived from observed evidence and the accepted portfolio goals.
LOCAL-UNKNOWN	Could exist on your Mac/desktop or in an agent session but is not observable from remote GitHub in this pass.

Primary conclusion

The portfolio has moved from a throughput problem to a closure problem. Remote activity is now extremely high. The bottleneck is converting parallel changes into coherent, installed, evidence-backed CVPs while reducing code and repository entropy without destructive loss.

Contents

- 1. Executive decision and snapshot delta
- 2. Define CVP: the installed-current-horizon finish line
- 3. Civis deep pass
- 4. Priority repository status and next closure targets
- 5. Soft deletion, tombstoning, and restore safety
- 6. Semantic compression: squish code without losing behavior
- 7. Recursive applets and federated hexagonal composition
- 8. Packaging, installability, polish, and third-party-like acceptance
- 9. QA/QE, journey evidence, and coverage requirements
- 10. Portfolio operating model and metrics
- 11. Execution waves / WBS
- 12. Agent handoff contract
- Appendices: evidence snapshot, decision rules, terminology

1. Executive decision and snapshot delta

The Sep 11 pass concluded that substantial development existed but was obscured by promotion branches, unmerged work, contradictory evidence, and local-state gaps. That diagnosis still matters, but the dominant problem has shifted.

Signal	Sep 11 baseline / prior finding	Sep 14 observation	Interpretation
Raw connected repo count	134 connected owner repos in the Sep 11 audit	138 repositories returned by the fresh owner search	Do not treat +4 as four new products; renames, restores, new targets and deletions all change the census.
Merged PR volume	113 owned-estate merges in the prior measured week window	493 owned-estate merged PRs since the Sep 11 cutoff; 163 are authored by KooshaPari	Throughput has accelerated sharply. PR count is now a poor optimization objective.
Breadth of remote movement	Broad but uneven	101 of 138 current repos show pushes since Sep 12	The estate is being touched at portfolio scale; much of it is lifecycle/CI/consolidation churn, not necessarily product value.
Civis	Runtime integration and correctness work	Persistence, visual polish, macOS .app bundling, cross-platform builds, nightly sandbox E2E	Civis is substantially closer to a user-testable CVP.
PhenoFabric	Contract/runtime groundwork	Runtime decomposition, benchmarks/fuzz scaffolding, auth/network CLI, NVMS adapter and CI work	Composition is becoming executable, but one real end-to-end multi-node product path remains the important proof.
Repository reduction	Mostly planning / candidate disposition	Large rename/archive/tombstone wave now active	Deletion safety and successor verification are now operational risks, not hypothetical governance questions.

Do not chase the old commit graph

The useful question is no longer “How many commits did the agents create?” It is “How many accepted capabilities reached the installed, verified, distributable state, and how much code/portfolio complexity was removed without semantic loss?”

1.1 What improved

- Civis now has direct movement toward a macOS app bundle and a clean sandbox playthrough rather than only library/runtime work.
- AgilePlus now has real desktop packaging work and a more honest self-assessment of its missing assurance layers.
- Tracera added real local-stack E2E against seeded production-like state and continued internal simplification.
- BytePort/NVMS work is actively moving duplicated NVMS responsibility into a canonical boundary and removing stale references.
- jcode is demonstrating exactly the high-parallelism benefits you described: rapid feature work and frequent decomposition of oversized TUI modules.
- Repository retirement has become concrete rather than theoretical; many repositories now carry explicit tombstone/archive naming or metadata.

1.2 What got more dangerous

- The much higher parallel write rate makes stale current-state assumptions and overlapping agents more damaging.
- Repository lifecycle state is inconsistent: name prefixes such as zz-/zz-archive and the GitHub archived flag do not map one-to-one.
- Some very recently active repositories are already archived, while other “archive”-named repositories remain writable. A name is not a lifecycle authority.

- High merge counts can hide low closure because code can move, be repackaged, or be cleaned up without producing a product you can actually install and use.
- jcode's concurrency benefit also creates integration hazards; its own recent history contains decomposition work followed by immediate fixes for broken concurrent edits or partially wired UI operations.

2. Define CVP: the installed-current-horizon finish line

MVP is the wrong term for what you are asking agents to do. You are not asking them to cut the idea down to the smallest hypothesis. You are asking them to freeze the currently accepted implementation horizon, stop broadening it, and make that current horizon actually usable. This audit calls that a Current Viable Product (CVP).

CVP definition

A CVP is the smallest deployable, installable, coherent, supportable, and sufficiently polished product that faithfully implements the currently accepted feature horizon. It may omit future roadmap features, but it may not hide already-accepted current behavior behind stubs, dead code, mocks, local-only setup, broken packaging, or unfinished UI.

Stage	Required outcome	Failure examples
C0 - Horizon freeze	Current accepted capabilities, non-goals, applets, platforms, and release target are explicit. New breadth is gated.	Agent keeps adding features while install path, persistence, or tests remain broken.
C1 - Clean build	Fresh checkout builds from declared inputs without sibling-repo magic, dirty files, or operator machine assumptions.	Works only from developer worktree; missing path deps; generated files required but absent.
C2 - Installable	Role-appropriate package installs on a clean target. macOS GUI means a real .app discoverable by LaunchServices/Spotlight.	Binary exists but app bundle is malformed; assets depend on repo cwd; dev toolchain required.
C3 - Useful journey	At least one full intended user task succeeds from installed artifact using real state and real adapters.	Only unit tests; demo path bypasses persistence/auth; UI opens but core action fails.
C4 - Durable	State survives restart; failures recover; upgrade/rollback/uninstall paths are known; no silent data loss.	Save works only in-process; crash corrupts state; migration has no rollback.
C5 - Polished	First-run, empty, loading, success, error, settings, keyboard/accessibility and visual states are intentionally designed.	Ugly debug chrome, clipped text, fake buttons, inconsistent spacing, incomplete screens.
C6 - Qualified / distributable	Required QA layers pass at accepted floors, journey evidence is captured, docs/release/provenance are complete, and artifact identity is tied to the tested SHA.	Passing PR checks but no tested installer; screenshot comes from dev build; release differs from tested artifact.

2.1 CVP is role-specific

Role	CVP artifact
Native/macOS GUI	Installed .app; DMG/ZIP as appropriate; correct bundle identifier, icon, storage paths and launch behavior.
Tauri desktop	Signed/ad-hoc local .app for internal CVP plus production signing/notarization path; frontend/backend lifecycle must be coherent.
CLI	Installable binary/package, shell completion/help, real terminal journey recorded through VHS, clean machine-readable output mode.
Library/SDK	Published or locally consumable package plus clean external consumer project and API/compatibility evidence.
Service	Deployable process/image with configuration, health, storage/migrations, shutdown/recovery, authentication and real client journey.
Harness/lab	Reproducible runner, pinned environment, one real benchmark/test corpus, output/evidence contract, and clean

Role	CVP artifact
	invocation from outside the source tree.
Historical donor/tombstone	No new product CVP. Completion means verified preservation, successor parity, consumer migration, restore proof and truthful terminal lifecycle.

3. Civis deep pass

Civis is the clearest repository where the new direction is working. Since the prior pass it has moved beyond runtime integration into the things that matter for your “put it in Spotlight and let me judge it” workflow: packaging, sandbox E2E, state durability, user-facing polish and cross-platform build work.

Area	Current observed movement	Assessment
macOS packaging	Current main adds a build-macos-app job that runs scripts/build-macos-app.sh and uploads Civis.app.	Strong CVP movement; still needs installed-artifact acceptance and release identity.
Cross-platform	Current build workflow now contains Windows, macOS and Linux civ-standalone jobs.	Good packaging breadth; artifact behavior must be tested, not just compiled.
E2E	Nightly sandbox job and --app smoke mode were added.	Correct direction; real native app journey and failure-state coverage remain important.
Persistence	Recent commits repair diplomacy, language/faction and institution/build-site/economic-focus round-trips.	Important durability work; should be consolidated into one restart/save/load journey.
Visual polish	Recent work includes command overlay, minimap, faction HUD, feed/notification polish and a 3D quality acceptance gate.	Meaningful, but aesthetics require installed-app visual review in addition to automated checks.
Mutation / assurance	A mutation ratchet is reported around a 65% CI-measured floor.	Transitional only; below your 85%+ independent assurance requirement.
Current Actions	At the sampled main head, GitHub reports 20 Actions runs: 14 successful and 6 failing.	Not globally clean. Failures need classification into required, environmental/advisory, or real defect.
Release	Published v0.5.0 contains CLI assets; the current .app work is newer.	Current app bundle is not yet proven as the released, installed CVP.

Civis decision

Stop treating the next feature as the primary milestone. The next milestone should be an artifact digest for one Civis.app, installed from outside the repo, launched like third-party software, completing the full save/restart/user journey while phenotype-journeys captures the real UI and all required QA gates are attached to that exact artifact.

3.1 Civis CVP closure sequence

1. Freeze current CVP horizon. Only accept new implementation work if it closes an already-accepted capability, a defect, a packaging requirement, or a measured quality/polish gap.
2. Build Civis.app from a clean checkout/release candidate. Store commit SHA, Cargo.lock digest, app bundle digest and build environment identity.
3. Install to ~/Applications or /Applications without depending on the repository. Verify LaunchServices registration, Spotlight discovery, icon, bundle identifier and launch via Finder/open.
4. Verify resource lookup and writable state use Application Support/appropriate user directories rather than cwd or source-tree paths.
5. Run one deterministic journey: first launch -> create/load world -> perform representative simulation actions -> save -> quit -> relaunch -> verify persisted state -> exercise reconnect/failure recovery.
6. Capture first-run, normal, selected-object, command palette, settings, empty/error/recovery and save/reload states through phenotype-journeys from the installed .app.

7. Run the full required QA matrix at the same candidate SHA: unit $\geq 85\%$, integration $\geq 85\%$, E2E $\geq 85\%$, mutation $\geq 85\%$ where applicable, critical obligations 100%, plus supported-platform acceptance.
8. Classify all current failing Actions. Required failures block CVP. Advisory/external failures must be explicitly labeled and cannot masquerade as green.
9. Perform visual/taste review separately from automated screenshot diffs. Fix clipping, density, spacing, hierarchy, debug affordances, inconsistent interaction patterns and low-quality default states.
10. Only then produce a CVP release and install that release artifact again. The release artifact, not a developer-built app, is what earns the “installed” state.

3.2 Civis acceptance record

```
CVP: Civis / current-horizon desktop
source_sha: <candidate>
artifact_sha256: <Civis.app archive/dmg digest>
installed_path: /Applications/Civis.app or ~/Applications/Civis.app
bundle_id: <verified>
spotlight_discoverable: true
clean_machine_launch: PASS
core_journey: PASS
save_restart_roundtrip: PASS
unit_coverage:  $\geq 85\%$ 
integration_coverage:  $\geq 85\%$ 
e2e_coverage:  $\geq 85\%$ 
mutation_score:  $\geq 85\%$  (eligible non-equivalent mutants)
critical_obligations: 100%
required_actions: PASS
visual_review: ACCEPTED
phenotype_journey_run: <id>
release_asset_matches_tested_artifact: true
```

4. Priority repository status and next closure targets

Area	Observed movement since prior pass	Current CVP diagnosis	Next proof
AgilePlus	Desktop now reads the real project DB; universal .app/DMG packaging and release work landed. Its own assessment reports 1,356 tests passing but calls out no coverage measurement, no transport tests, no performance baseline and no full E2E.	Near packaging CVP, not qualified CVP. Self-assessment is usefully honest.	Install current app outside source tree; prove project discovery + read/write workflow, coverage, transport, perf and full E2E on the tested artifact.
sharecli	Agent/tool detection, tray/popover polish, responsive UI and large test/coverage pushes.	Quality is moving toward threshold; package/product acceptance remains the important next step.	Native coverage report at current head; packaged tray/CLI artifact; supervise a real workload, fail/restart/recover, and verify no foreground contention regression.
Tracera	Real seeded local-stack E2E, DB integration tests, frontend test prep, branch-protection repair and internal simplification.	Good correctness movement. Chat being down means remote state should become the handoff source rather than session memory.	Re-establish owner, pick one deployment mode, prove persistent datastore/auth/MCP/web journey, then package/deploy the same candidate.
PhenoFabric	Runtime module decomposition, benchmarks/fuzz scaffolding, auth/network CLI work, settings persistence and NVMS adapter repair.	More than a spec repo now, but still needs product composition proof.	One real two-node path: discovery/auth -> route -> surface/data operation -> node failure -> replan/recovery, with latency/resource evidence and packaged endpoints.
terminal-fabric	New private repo: Windows Terminal UIA multi-tab capture, web serving, pane cache, privacy fix, module decomposition.	Useful observability/control substrate. Treat terminal content as sensitive by default.	Installable local service/agent, auth and redaction policy, restart persistence, multi-terminal capture journey, resource budget.
jcode	Very high activity: swarm ops, DAG/filter/rename UI, terminal/shell integration, protocol metrics, module splits and persistent fork-depth guard.	Powerful harness with integration churn. Recent history also contains quick follow-up repairs after concurrent edits/partially wired operations.	Keep high-concurrency work, but restrict destructive fleet/repo lifecycle operations behind transactional APIs and a preview/lease model.
BytePort + nanovms	NVMS implementation removed	Exactly the desired semantic-	Consumer graph + clean build +

Area	Observed movement since prior pass	Current CVP diagnosis	Next proof
	from BytePort after repoint; CI stale references removed; FFI responsibility migrated into nanovms; boundary ADRs updated.	compression direction if consumer parity is real.	BytePort/NVMS end-to-end after deletion; restore test; eliminate remaining duplicate implementation only after parity.
Portage	Recent work is mostly blocking-gate repair, lint cleanup and CI conflict repair.	Maintenance has dominated product proof.	Run one real adapter/evaluation job from clean install; measure output and failure handling; then decide which remaining gate work is actually CVP-blocking.
PhenoLab	vLLM serve/config work, stress demo, attestations and Tracera absorption semantics.	Lab CVP should be reproducibility, not a GUI app. Environmental failures still appear in recent history.	One pinned benchmark bundle reproducible on a clean runner with results/provenance; no skipped contract checks counted as delivery.
hwLedger	Recent commits mostly make quality gates genuinely blocking and repair workflow formatting.	Good QE cleanup, little recent product proof visible.	Install/run hardware probe, persist/query inventory, exercise at least two adapters/hardware classes and failure path, then package.
PhenoMLX / hfscope	Both are now archive-labelled/archived; hfscope had feature work as recently as Sep 11. PhenoMLX previously had signed/notarized Apple work.	High-risk lifecycle mismatch with the AI-area chat you described.	Do not delete. Verify active sessions/consumers/successors and absorption parity; retain tombstone until no active work remains and restore drill passes.
phenotype-sdks	New consolidation repo imported SDK/observability content.	Potentially good consolidation, but “SDK” must not become another generic attic.	Define package boundaries and consumers; distinguish SDK surface from runtime observability service/operations ownership.
koosha-phenotype	Vercel build/performance work, image compression, JS minification and large CLS improvement are visible.	Healthy web-delivery focus; likely separate from product runtimes.	Deploy current build, run Lighthouse/visual/accessibility on production URL, verify no Playwright/build-only hacks leak into runtime.

5. Soft deletion, tombstoning, and restore safety

The current retirement wave is now large enough that repository deletion must be treated as a data-migration and provenance operation, not a cleanup command. The user-specified “never delete a fork-network member by default” rule is sound and should be stricter than agent discretion.

Hard rule

No agent or subagent gets direct authority to permanently delete a repository that participates in a fork network. Default outcome is tombstone + preserved repository + retention. Permanent deletion is a separately authorized lifecycle transition after evidence and retention gates.

State	Allowed actions	Exit gate
ACTIVE	Normal development/release.	Accepted disposition exists.
RETIRING	Freeze new feature breadth; inventory refs/releases/issues/packages/consumers; build successor map.	Preservation + successor plan complete.
TOMBSTONED	Rename/prefix as desired, prominent pointer, disable routine automation, preserve refs/history. Do not delete.	Successor parity + consumer migration + restoration artifact verified.
QUARANTINED	Optional GitHub archive/read-only after checking active worktrees/agents and release consumers.	Retention window elapsed with no restore event.
DELETE_ELIGIBLE	Non-fork-network only; no active consumer/session; restore drill passed; human/operator approval recorded.	Explicit delete action by restricted lifecycle operator.
DELETED	Only after all previous gates; external provenance/bundle remains independently verifiable.	N/A

5.1 Default retention

Use 60 days as the default tombstone/quarantine period for this cleanup wave. A shorter period is an exception requiring explicit operator approval; fork-network repositories default to indefinite non-deletion unless a later human decision establishes that deletion is actually useful.

5.2 Retirement manifest - required per repository

- GitHub repository numeric ID, original and current name, visibility, default branch, archived flag and fork/network relationship.
- All relevant refs/tags/releases/packages plus preservation bundle/hash and the exact object IDs needed to reconstruct history.
- Open issues/PRs/discussions and any operational data not contained in Git itself.
- Successor repository and source->target path/commit mapping for every absorbed capability.
- Consumer scan: code imports, package consumers, CI references, deployment references, docs links, submodules and active agent/session ownership.
- Parity evidence: source capability inventory versus successor implementation/tests/artifacts.
- Restore procedure and an actual restore drill before permanent deletion eligibility.
- Retention-until timestamp, lifecycle approver, exceptions and reason.

5.3 Current estate warning

The current account demonstrates why names cannot carry lifecycle semantics. Some repositories have zz-archive-style names while remaining writable/unarchived; some archived repositories do not follow that naming pattern; and some tombstone-prefixed repositories are still active transition surfaces. Lifecycle state must live in a machine-readable record keyed by repository ID, with the name as a presentation convention only.

6. Semantic compression: squish code without losing behavior

The “squish the water out” model is directionally correct, but the safe objective is semantic compression rather than line-count reduction. The system should become easier to understand, modify, test and compose while preserving every accepted behavior and quality property.

Semantic compression principle

Remove representational redundancy, accidental abstractions, dead paths, duplicate implementations, duplicate repositories, hand-written boilerplate and needless indirection. Preserve accepted semantics, contracts, failure behavior, performance bounds, user experience, compatibility and provenance.

Finding	Disposition
Duplicate behavior with one canonical implementation already verified	Delete duplicate implementation after consumer migration; preserve provenance and compatibility adapter only where needed.
Dead code for capability still in current CVP intent	Do not leave dead. Either reconnect/implement it now or formally remove it from the accepted CVP through an explicit decision.
Dead code for later roadmap capability	Move the intent/spec/research/evidence out of executable source; remove unreachable implementation. Re-implement when the roadmap gate opens.
Superseded implementation	Delete from active tree; keep commit/tag/history and migration record.
Unknown/orphan code	Characterize with call graph, tests, blame/intent/history and consumer search before deletion.
Generated/vendor code	Exclude from cognitive-size metrics; regenerate from canonical source or vendor intentionally with provenance.
Huge module with mixed responsibilities	Split by stable domain/port/adaptor boundaries, not arbitrary

Finding	Disposition
	file-size targets. File-size limits can be a lint signal, never the architecture itself.

6.1 Required before/after measures

Measure	Desired direction
Hand-authored LOC and token count	Down where semantics are unchanged.
Number of public concepts/types/commands user must understand	Down or stable.
Duplicate capability implementations	Down toward one authority per semantic responsibility.
Dependency edges and cycles	Down; cycles ideally zero across bounded contexts.
Largest module cognitive load	Down through domain decomposition.
Behavioral/contract coverage	Stable or up; critical obligations remain complete.
Unit/integration/E2E/mutation quality	Stable or up; never traded for shorter code.
Binary size/startup/memory/latency	Must not regress outside declared tolerances.
Build/release complexity	Down where possible.

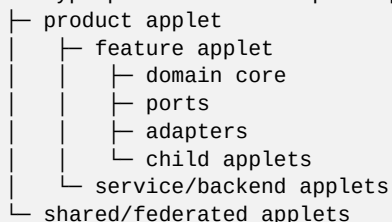
6.2 Compression pass per repository

1. Freeze current behavior contract and collect independent characterization tests before structural deletion.
2. Build duplicate clusters by symbols, APIs, behavior, schemas, tests and history - not filename similarity alone.
3. Collapse duplicate code into the nearest correct domain owner; add a compatibility facade only when real consumers need it.
4. Remove unreachable current-horizon code only after deciding whether it is missing implementation or obsolete semantics.
5. Reduce dependency fan-out and public exports. Internal implementation detail should remain internal.
6. Split modules only along domain/port/adaptor or applet boundaries. Avoid “one file per idea” fragmentation.
7. Run semantic equivalence: same contract tests, negative tests, benchmarks and journey outputs before/after.
8. Record net simplification and any intentionally changed semantics in ADR/spec traceability.

7. Recursive applets and federated hexagonal composition

The recursive applet idea is a useful refinement of the current hexagonal direction. The key is to make “applet” a compositional runtime/software boundary, not automatically a repository boundary.

Phenotype platform / workspace applet



Any composite may expose a reduced set of child capabilities as its own ports.
A composite is therefore also an applet at the next level.

7.1 Applet contract

Field	Purpose
identity/version	Stable applet identity, compatibility version and provenance.
capabilities	What semantic operations the applet offers; capability IDs are

Field	Purpose
	more important than package names.
ports	Typed commands, queries, events, streams, files, UI surfaces and resource interfaces.
adapters	OS, database, provider, hardware, transport and presentation implementations.
state	Owned durable/ephemeral state, schema/version, migration and consistency rules.
permissions	Least-privilege capability grants and destructive-operation restrictions.
resources	CPU/GPU/memory/storage/network/real-time constraints; feeds PhenoFabric scheduling where applicable.
children	Optional child applets and federation policy.
health/lifecycle	Startup readiness, degradation, failure/restart semantics, update/rollback.
surfaces	Optional UI/CLI/API/audio/video/terminal surfaces exposed to users or other applets.

7.2 Composition rules

- An applet may be embedded in-process, run as a process/service, live in a VM/container, or execute on another node. Location is an adapter/deployment concern, not its semantic identity.
- A parent sees a child through declared capabilities/ports, not internal modules. This preserves hexagonal boundaries at every recursive level.
- Federation is dynamic only when lifecycle/state/security constraints permit it. “Optional module” must not mean “unchecked plugin can mutate everything.”
- Composite applets re-export only intentional capabilities. Do not leak every child API upward.
- A product CVP is a named composition profile of applets plus adapters and policy. This allows one codebase to produce small or rich product assemblies without semantic forks.
- Repository extraction is justified by independent release/consumer/security/lifecycle boundaries, not merely because an applet exists.

7.3 Why this helps simplification

Recursive applets give the code-compression work a destination. Instead of retaining multiple “maybe useful later” implementations, preserve the capability contract, select one current implementation, and make optionality explicit in the composition profile. This converts hidden dead code into either a live applet, a documented future capability, or removed implementation.

8. Packaging, installability, polish, and third-party-like acceptance

Your “put the .app in Spotlight” requirement should become a formal product gate. It is one of the best ways to force agents out of repo-centric thinking and into product-centric thinking.

8.1 macOS Spotlight CVP gate

- Bundle installs outside the repository and launches with no developer shell, cwd, node_modules, Cargo target directory or sibling checkout dependency.
- CFBundleIdentifier, display name, semantic version/build version, icon, Info.plist and architecture are valid.
- Application data, caches and logs use platform-appropriate directories; nothing durable is written inside the application bundle.
- LaunchServices registers the app; Spotlight can find it; ``open -b <bundle-id>`` or Finder launch works.
- First launch handles permissions and missing configuration intentionally; no silent terminal-only error path.

- Quit/relaunch and window/state restoration behave deliberately; menu/keyboard conventions feel native enough for the product category.
- The artifact under review is the same digest that passed E2E and visual capture.
- For external/public release, code signing/notarization/update channel requirements are satisfied; internal CVP may use an explicitly labelled local/ad-hoc path.

8.2 “Ugly app” prevention

Layer	Mandatory review
Information architecture	Clear primary task, visible state, no unexplained debug/engineering concepts, no duplicate controls.
Layout	Spacing scale, alignment, density, clipping, window resizing, min/max sizes, panel hierarchy.
Typography/color	Consistent roles, contrast, semantic status colors, dark/light behavior if supported, no accidental default styles.
Interaction	Keyboard focus, shortcuts, hover/selected/disabled states, destructive confirmation, progress, cancellation, undo where appropriate.
System integration	Menu bar, tray/dock behavior, notifications, file open/save, clipboard, deep links, platform permissions as applicable.
State coverage	First-run, empty, loading, normal, partial, error, offline/degraded, recovery and upgraded/migrated state.
Performance feel	Startup, input latency, scrolling/rendering, background CPU/GPU, no perceptible stalls under intended workloads.

8.3 phenotype-journeys evidence

Every GUI CVP should be exercised from the installed artifact in a private/isolated native environment. Record raw screenshots/video, actions, artifact SHA, OS/display profile and assertions. Store raw evidence separately from edited marketing assets. A beautiful screenshot from a dev-only path cannot substitute for installed-product evidence.

9. QA/QE, journey evidence, and coverage requirements

Independent floors

For retained products, unit, integration, E2E and other applicable assurance families each target $\geq 85\%$ against a fixed, reviewed denominator. Critical safety, data-integrity, security, packaging and promised-compatibility obligations are 100%. A combined 95% report cannot hide a 60% integration or E2E layer.

Gate	Minimum acceptance
Unit	$\geq 85\%$ executable/branch/function coverage where measurable + $\geq 85\%$ approved behavioral obligations; critical obligations 100%.
Integration	$\geq 85\%$ boundary/protocol/storage/process/platform obligations and structural coverage under its own profile.
E2E	$\geq 85\%$ declared user/consumer journeys and instrumented implementation under the E2E profile; critical journeys 100%.
Mutation	$\geq 85\%$ killed/covered eligible non-equivalent mutants where mutation is applicable; surviving critical mutants block.
Fuzz/property	All critical parser/protocol/state-machine targets have explicit campaigns/invariants and no unresolved critical findings.
Performance	Representative cold/warm/loaded cases, p50/p95/p99 where meaningful, and no undeclared regression beyond budget.
Compatibility	Accepted OS/arch/dependency/configuration matrix; unknown/skipped is not pass.
Packaging	Clean-install and clean-upgrade/rollback E2E from the produced artifact.
Visual/accessibility	Automated checks plus human/device review for qualities

Gate	Minimum acceptance
	automation cannot establish.

9.1 Important current gaps

- Civis's reported 65% mutation ratchet is useful as a transition mechanism but does not meet the 85% portfolio acceptance floor.
- Civis current head has 14 successful and 6 failing Actions runs. Required failures must be resolved; advisory/environmental failures need explicit classification.
- AgilePlus's own recent assessment identifies missing coverage measurement, transport tests, performance baselines and full E2E. That is exactly the evidence needed before "desktop app complete."
- Test counts are not coverage. "3,676 tests pass" can coexist with major untested surfaces or weak oracles.
- Any gate that skips because the implementation is absent is measuring development state, not satisfying the product contract.

10. Portfolio operating model and metrics

At the present throughput, PR/commit count should be demoted to telemetry. Optimize closure velocity and complexity reduction instead.

Metric	Why it matters
Installed CVPs / week	Direct measure of products reaching the state where you can actually use them and give feedback.
Median time: CVP freeze -> installed artifact	Measures ability to convert current code into product reality.
Capabilities moved to Verified/Released/Adopted	Separates implementation churn from real maturation.
Repository terminal dispositions verified	Tracks portfolio shaping without rewarding premature deletion.
Restore-loop count after retirement action	Direct defect metric for unsafe deletion/absorption. Should trend to zero.
Semantic compression: duplicate implementations removed with parity	Measures "squishing" without rewarding blind LOC deletion.
Open current-head required failures	Shows whether QA debt is falling.
Unintegrated branch age / dirty work age	Shows hidden/local work accumulating without landing.
Installed-artifact feedback defects	Reveals gaps in automated tests and polish review.
Agent WIP by capability slice	Controls concurrency without forcing one-chat-per-repo.

10.1 State model per capability slice

WORKING	local/dirty implementation exists
PUBLISHED	remote branch/PR contains it
INTEGRATED	accepted integration/default branch contains it
VERIFIED	required independent gates passed at that exact revision
PACKAGED	role-appropriate distributable artifact produced
INSTALLED	artifact installed/consumed outside source tree
RELEASED	authorized release/deployment published
ADOPTED	real intended user/consumer successfully uses it

These states are orthogonal to repository lifecycle (ACTIVE/RETIRING/TOMBSTONED/etc.).

10.2 Agent concurrency rule

Keep the benefits of jcode/ForgeCode parallelism, but coordinate around capability slices and integration ownership. A repo may have multiple chats if slices are disjoint. Every slice still needs one integration owner and one current acceptance record. Destructive lifecycle operations are not delegated to ordinary subagents.

11. Execution waves / WBS

Wave	Objective	Representative tasks	Exit
W0 - Truth + freeze	Stop current-state ambiguity and destructive drift.	Refresh current heads; bind chats to capability slices; freeze CVP horizons; quarantine deletions; classify Actions failures; create retirement manifests.	Every active slice has owner, target artifact, exact revision, acceptance gaps and next task.
W1 - First installed CVPs	Convert highest-value mature apps into real installable products.	Civis.app install/restart journey; AgilePlus desktop install/project workflow; ShareCLI packaged supervision journey.	At least 2 installed CVPs with artifact-bound evidence and feedback capture.
W2 - Composition proofs	Prove platform/runtime systems compose.	PhenoFabric two-node path; Tracera real datastore/auth/client; terminal-fabric secure capture service; BytePort<->nanovms consumer parity.	Each system has one full real-data E2E path, not only contracts.
W3 - Semantic compression	Reduce code/repo complexity under equivalence tests.	Duplicate clusters; dead-code disposition; module/appet boundaries; remove migrated copies; dependency-cycle reduction.	Net complexity reduction with preserved behavioral evidence.
W4 - Lab/infra CVPs	Move maintenance-heavy repos to useful runnable outcomes.	Portage real evaluation job; PhenoLab reproducible benchmark; hwLedger install/probe/query; SDK clean consumer.	Each retained repo demonstrates its independent user/consumer value.
W5 - Portfolio terminalization	Safely finish retirement/consolidation.	Retention timers; restore drills; archive/tombstone consistency; successor links; public/docs routing.	Terminal states are reversible where policy requires and no active consumer is orphaned.

11.1 Admission policy

Within a CVP-focused repository, do not start a new feature simply because a worker becomes available. Admit new breadth only when it is required by the accepted current horizon, fixes an observed defect, unblocks packaging, or closes an evidence/polish gap. Otherwise, put it in the next-horizon backlog and keep compressing toward installability.

12. Agent handoff contract

The following compact contract should be appended to active implementation chats. A separate Markdown copy is delivered with this report.

```

You own a capability slice, not merely a branch or list of files.
Your parent outcome is the repository's accepted CVP or retirement target.

1. Reconcile current local + remote state before changing anything.
2. Freeze the current CVP horizon. Do not expand breadth unless it closes an accepted gap.
3. Prefer semantic compression: one canonical behavior, fewer concepts, fewer duplicate paths.
4. Dead code is not allowed to linger unexplained:
  - required now -> wire/implement it;
  - future only -> preserve intent/spec, remove executable dead path;
  - superseded -> delete with history/provenance retained;
  - unknown -> investigate before deletion.
5. Preserve hexagonal boundaries and express optional composition as recursive applets.
6. For apps, do not stop at 'builds'. Produce the role-appropriate installed artifact.
  macOS GUI target: .app outside the repo, LaunchServices/Spotlight discoverable.
7. Run real installed-artifact journeys and capture evidence via phenotype-journeys.
8. Independent QA floors: unit/integration/E2E/other applicable families >=85%; critical obligations 100%.
9. Do not claim product completion from test counts, screenshots, PR merge, or current-head CI alone.
10. Classify every remaining blocker by owner and exact unblock action.

```

11. Never permanently delete a fork-network repository. Ordinary agents may prepare retirement evidence and tombstones, not irreversible deletion.
12. Report separately: working, published, integrated, verified, packaged, installed, released, adopted.

Finish the smallest coherent CVP/retirement outcome before taking another slice.

Appendix A - Evidence snapshot and observations

Observation	Value / evidence
Fresh repo census	138 repositories returned by current owner search. Count is a snapshot, not a steady-state target.
Recently pushed breadth	101 of the 138 current repositories were returned by a pushed:>=2026-09-12 search.
Merged PRs since prior cutoff	493 merged PRs in owned estate after the prior Sep 11 cutoff; 163 were authored by KooshaPari.
Civis latest sampled head	c1302e934fb77d15b70b848a01e1c1bca59e23ba - cross-platform builds, Civis.app job and E2E sandbox expansion.
Civis Actions at that head	20 runs observed: 14 success, 6 failure. First sampled failure is OpenSSF Scorecard; all failures still require per-gate classification.
Civis release state	Published v0.5.0 has CLI platform assets; current app-bundle work is newer than that release.
AgilePlus recent self-assessment	Reports packaging progress and 1,356 tests, while explicitly identifying missing coverage, transport, performance and full E2E evidence.
BytePort/NVMS boundary	BytePort removed local NVMS implementation after repoint; nanovms received FFI responsibility and boundary documentation.
Current lifecycle churn	Many current repositories now use zz-/zz-archive naming and/or GitHub archived state, with visible mismatches between name and archived metadata.

Appendix B - Decision rules

- A repository is not “done” because a PR merged; the CVP/repository target is done only when its role-specific terminal acceptance record is complete.
- A repository is not “safe to delete” because its code appears copied elsewhere; successor parity, consumers, provenance and restore must be proved.
- A smaller codebase is not automatically better; simplification is accepted only when semantics/quality are preserved or intentionally changed through an explicit decision.
- An applet is not automatically a repository. Prefer package/module composition until independent release, ownership, security or consumer boundaries justify extraction.
- A high test count is not broad assurance. Use fixed denominators and independent unit/integration/E2E/other-family gates.
- A beautiful marketing asset is not product evidence. Capture raw installed-product evidence first; derive presentation assets second.
- A local build is not a release. Tested revision, package digest, installed artifact and published artifact identities must be traceable.

Appendix C - Terminology

Term	Definition
CVP	Current Viable Product: deployable/useful implementation of the accepted current horizon, without cutting away already-

Term	Definition
	accepted current features.
Semantic compression	Reduction of code/concept/repository complexity while preserving accepted semantics and measured quality.
Applet	Recursively composable hexagonal capability unit with declared ports, state, permissions, resources and optional surfaces.
Tombstone	Preserved repository or lifecycle record that prevents new ownership while keeping provenance and restore capability.
Terminal disposition	Verified long-lived repository state: retained product/foundation, strategic fork, historical archive/tombstone, absorbed donor, or intentional retirement.